

DBMS CASE STUDY

Quick Commerce System – Customer Management & Product Management

1. Problem Definition

Quick commerce (Q-commerce) platforms deliver essential products such as groceries, medicines, and daily-use items within a very short time frame (10–30 minutes). In such systems, **managing customer data and product data accurately and efficiently** is critical for smooth operations.

Currently, many small and medium quick commerce setups rely on:

- Manual records
- Unorganized spreadsheets
- Separate systems for customers and products

This leads to:

- Data redundancy and inconsistency
- Difficulty in tracking customer orders and purchase history
- Poor inventory control and product availability errors
- Inefficient customer service due to missing or incorrect information

Hence, there is a need for a **centralized relational database system** that can manage **customer information and product information** in an organized, secure, and scalable manner.

2. Objectives of the System

The main objectives of the proposed Quick Commerce Database System are:

1. To maintain **accurate and structured customer information** such as customer ID, name, contact details, address, and registration date.
2. To store and manage **product details** including product ID, product name, category, price, stock quantity, and availability status.
3. To establish a **relationship between customers and products** through orders for tracking purchases.
4. To reduce data redundancy using **proper normalization and relational design**.
5. To enable efficient **SQL operations in SQL*Plus**, including:
 - Data insertion and updates

- Searching and filtering records
- Aggregate queries for analysis (e.g., total products, active customers)
- 6. To support future expansion such as order management, delivery tracking, and payment modules.

3. Practical Scenario Relevance

This case study reflects a **real-world quick commerce scenario**, similar to platforms operating in urban areas where:

- Customers register on the app to place instant orders.
- Products are stored in nearby warehouses or dark stores.
- Each product has limited stock that must be updated after every order.
- Customer purchase history helps in understanding demand patterns.

Example scenario:

- A customer places an order for grocery items.
- The system checks product availability.
- Customer and product records are updated in real time.

Such operations require a **robust database system**, making this case study highly relevant for practical DBMS implementation.

4. System Scope Identified

Included in Scope

- Customer Management
 - Customer registration
 - Storage of customer details
 - Viewing and updating customer information
- Product Management
 - Product catalog maintenance
 - Stock quantity tracking
 - Product price and category management
- Database Operations using SQL*Plus
 - Table creation with primary and foreign keys
 - Data insertion (minimum 5 records per table)
 - Execution of SQL queries for retrieval and analysis

2. ER Diagram & Design

Quick Commerce System

2.1 Identification of Entities

Based on the case study and system scope, the following **core entities** are identified:

1. **Customer**
 2. **Product**
 3. **Order**
 4. **Order_Item** (Associative / Weak Entity)
-

2.2 Entity Description with Attributes

1. Customer

Represents users who place orders on the quick commerce platform.

Attributes:

- **customer_id** (Primary Key)
 - customer_name
 - phone_number
 - email
 - address
 - registration_date
-

2. Product

Represents items available for instant delivery.

Attributes:

- **product_id** (Primary Key)
- product_name
- category
- price
- stock_quantity
- availability_status

3. Order

Represents a purchase transaction made by a customer.

Attributes:

- **order_id** (Primary Key)
 - order_date
 - order_status
 - total_amount
 - **customer_id** (Foreign Key)
-

4. Order_Item

Represents products included in a specific order
(Resolves the many-to-many relationship between Order and Product).

Attributes:

- **order_item_id** (Primary Key)
 - quantity
 - price_at_order
 - **order_id** (Foreign Key)
 - **product_id** (Foreign Key)
-

2.3 Relationships & Cardinalities

1. Customer — Places — Order

- **Relationship:** Customer *places* Order
 - **Cardinality:**
 - One Customer → Many Orders
 - Each Order → Exactly One Customer
 - **Type:** One-to-Many (1:N)
-

2. Order — Contains — Product

- **Relationship:** Order *contains* Product
- **Cardinality:**

- One Order → Many Products
 - One Product → Can appear in Many Orders
 - **Type:** Many-to-Many (M:N)
 - **Resolved Using:** Order_Item entity
-

3. Order — Has — Order_Item

- **Cardinality:**
 - One Order → Many Order_Items
 - Each Order_Item → One Order
 - **Type:** One-to-Many (1:N)
-

4. Product — Appears In — Order_Item

- **Cardinality:**
 - One Product → Many Order_Items
 - Each Order_Item → One Product
 - **Type:** One-to-Many (1:N)
-

2.4 ER Diagram Notation (Exam-Oriented)

- **Rectangles** → Entities (Customer, Product, Order, Order_Item)
 - **Ellipses** → Attributes
 - **Underlined attributes** → Primary Keys
 - **Diamonds** → Relationships (Places, Contains, Has)
 - **Double rectangle** → Weak/Associative entity (Order_Item)
 - **Crow's Foot / 1:N notation** → Cardinalities
-

2.5 Design Justification

- The ER model is **conceptually correct and complete**
- All **entities and attributes are logically identified**
- **Primary and foreign keys** ensure referential integrity
- **Many-to-many relationship** is properly resolved
- Design supports **SQL*Plus table creation without redundancy**
- Easily extendable for delivery, payments, or reviews

3. Schema Design & Data Insertion

Quick Commerce System

3.1 Schema Design Overview

The database schema is designed based on the ER diagram and follows **relational database principles** such as:

- Proper normalization
- Use of **Primary Keys** and **Foreign Keys**
- Appropriate **data types** suitable for SQL*Plus
- Logical attribute grouping

The schema consists of **four tables**:

1. CUSTOMER
 2. PRODUCT
 3. ORDERS
 4. ORDER_ITEM
-

3.2 Table Structures (SQL*Plus Compatible)

1. CUSTOMER Table

```
CREATE TABLE customer (  
    customer_id NUMBER PRIMARY KEY,  
    customer_name VARCHAR2(50) NOT NULL,  
    phone_number VARCHAR2(15) UNIQUE,  
    email VARCHAR2(50),  
    address VARCHAR2(100),  
    registration_date DATE  
);
```

2. PRODUCT Table

```
CREATE TABLE product (  
    product_id NUMBER PRIMARY KEY,  
    product_name VARCHAR2(50) NOT NULL,  
    category VARCHAR2(30),  
    price NUMBER(8,2) CHECK (price > 0),  
    stock_quantity NUMBER CHECK (stock_quantity >= 0),  
    availability_status VARCHAR2(15)  
);
```

3. ORDERS Table

```
CREATE TABLE orders (  
    order_id NUMBER PRIMARY KEY,  
    order_date DATE,  
    order_status VARCHAR2(20),  
    total_amount NUMBER(10,2),  
    customer_id NUMBER,  
    CONSTRAINT fk_customer  
    FOREIGN KEY (customer_id)  
    REFERENCES customer(customer_id)  
);
```

4. ORDER_ITEM Table

```
CREATE TABLE order_item (  
    order_item_id NUMBER PRIMARY KEY,  
    quantity NUMBER CHECK (quantity > 0),  
    price_at_order NUMBER(8,2),  
    order_id NUMBER,  
    product_id NUMBER,  
    CONSTRAINT fk_order  
    FOREIGN KEY (order_id)  
    REFERENCES orders(order_id),  
    CONSTRAINT fk_product  
    FOREIGN KEY (product_id)  
    REFERENCES product(product_id)  
);
```

3.3 Data Insertion (Minimum 5 Records per Table)

Insert Data into CUSTOMER

```
INSERT INTO customer VALUES (1, 'Amit Sharma', '9876543210',  
'amit@gmail.com', 'Mumbai', SYSDATE);  
INSERT INTO customer VALUES (2, 'Neha Patil', '9123456780',  
'neha@gmail.com', 'Pune', SYSDATE);  
INSERT INTO customer VALUES (3, 'Rahul Verma', '9988776655',  
'rahul@gmail.com', 'Delhi', SYSDATE);  
INSERT INTO customer VALUES (4, 'Sneha Joshi', '9012345678',  
'sneha@gmail.com', 'Nagpur', SYSDATE);  
INSERT INTO customer VALUES (5, 'Karan Mehta', '9090909090',  
'karan@gmail.com', 'Ahmedabad', SYSDATE);
```

Insert Data into PRODUCT

```
INSERT INTO product VALUES (101, 'Milk', 'Dairy', 50, 100,  
'Available');  
INSERT INTO product VALUES (102, 'Bread', 'Bakery', 40, 60,  
'Available');  
INSERT INTO product VALUES (103, 'Rice', 'Grocery', 70, 200,  
'Available');  
INSERT INTO product VALUES (104, 'Eggs', 'Poultry', 6, 300,  
'Available');
```

```
INSERT INTO product VALUES (105, 'Soap', 'Personal Care', 30, 150, 'Available');
```

Insert Data into ORDERS

```
INSERT INTO orders VALUES (1001, SYSDATE, 'Delivered', 250, 1);
INSERT INTO orders VALUES (1002, SYSDATE, 'Pending', 180, 2);
INSERT INTO orders VALUES (1003, SYSDATE, 'Delivered', 320, 3);
INSERT INTO orders VALUES (1004, SYSDATE, 'Cancelled', 90, 4);
INSERT INTO orders VALUES (1005, SYSDATE, 'Delivered', 150, 5);
```

Insert Data into ORDER_ITEM

```
INSERT INTO order_item VALUES (1, 2, 50, 1001, 101);
INSERT INTO order_item VALUES (2, 1, 40, 1002, 102);
INSERT INTO order_item VALUES (3, 3, 70, 1003, 103);
INSERT INTO order_item VALUES (4, 6, 6, 1004, 104);
INSERT INTO order_item VALUES (5, 2, 30, 1005, 105);
```

3.4 Schema Validation

- ✓ All tables have **Primary Keys**
 - ✓ Foreign keys maintain **referential integrity**
 - ✓ Correct data types used for SQL*Plus
 - ✓ **Minimum 5 meaningful records** inserted per table
 - ✓ Constraints ensure data accuracy
-

CUSTOMER TABLE:-

```
SQL> select * from customer;
```

CUSTOMER_ID	CUSTOMER_NAME	PHONE_NUMBER	EMAIL	ADDRESS	REGISTRATION_DATE
1	Amit Sharma	9876543210	amit@gmail.com	Mumbai	25-JAN-26
2	Neha Patil	9123456780	neha@gmail.com	Pune	25-JAN-26
3	Rahul Verma	9988776655	rahul@gmail.com	Delhi	25-JAN-26
4	Sneha Joshi	9012345678	sneha@gmail.com	Nagpur	25-JAN-26
5	Karan Mehta	9090909090	karan@gmail.com	Ahmedabad	25-JAN-26

```
SQL> desc customer;
```

Name	Null?	Type
CUSTOMER_ID	NOT NULL	NUMBER
CUSTOMER_NAME	NOT NULL	VARCHAR2(15)
PHONE_NUMBER		VARCHAR2(15)
EMAIL		VARCHAR2(15)
ADDRESS		VARCHAR2(10)
REGISTRATION_DATE		DATE

PRODUCT TABLE:-

```
SQL> select * from product;
```

PRODUCT_ID	PRODUCT_NAME	CATEGORY	PRICE	STOCK_QUANTITY	AVAILABILITY_STATUS
101	Milk	Dairy	50	100	Available
102	Bread	Bakery	40	60	Available
103	Rice	Grocery	70	200	Available
104	Eggs	Poultry	6	300	Available
105	Soap	Personal Care	30	150	Available

```
SQL> desc product;
```

Name	Null?	Type
PRODUCT_ID	NOT NULL	NUMBER
PRODUCT_NAME	NOT NULL	VARCHAR2(15)
CATEGORY		VARCHAR2(15)
PRICE		NUMBER(8,2)
STOCK_QUANTITY		NUMBER
AVAILABILITY_STATUS		VARCHAR2(10)

ORDERS TABLE:-

```
SQL> select * from orders;
```

ORDER_ID	ORDER_DATE	ORDER_STATUS	TOTAL_AMOUNT	CUSTOMER_ID
1001	25-JAN-26	Delivered	250	1
1002	25-JAN-26	Pending	180	2
1003	25-JAN-26	Delivered	320	3
1004	25-JAN-26	Cancelled	90	4
1005	25-JAN-26	Delivered	150	5

```
SQL> desc orders;
```

Name	Null?	Type
ORDER_ID	NOT NULL	NUMBER
ORDER_DATE		DATE
ORDER_STATUS		VARCHAR2(12)
TOTAL_AMOUNT		NUMBER(10,2)
CUSTOMER_ID		NUMBER

ORDER ITEM:-

```
SQL> select * from order_item;
```

ORDER_ITEM_ID	QUANTITY	PRICE_AT_ORDER	ORDER_ID	PRODUCT_ID
1	2	50	1001	101
2	1	40	1002	102
3	3	70	1003	103
4	6	6	1004	104
5	2	30	1005	105

```
SQL> desc order_item;
```

Name	Null?	Type
ORDER_ITEM_ID	NOT NULL	NUMBER
QUANTITY		NUMBER
PRICE_AT_ORDER		NUMBER(8,2)
ORDER_ID		NUMBER
PRODUCT_ID		NUMBER

4. SQL Queries Demonstration

Quick Commerce System

4.1 Objective of SQL Queries

The objective of this section is to demonstrate the effective use of **SQL commands** for:

- Database definition
- Data manipulation
- Data retrieval
- Data analysis
- Enforcing constraints

As per the rubric, **at least 10 SQL queries** covering **minimum 5 functionalities** are demonstrated.

4.2 SQL Functionalities Covered

- ✓ DDL (CREATE, ALTER)
 - ✓ DML (INSERT, UPDATE, DELETE)
 - ✓ Aggregate Functions
 - ✓ GROUP BY & HAVING
 - ✓ Joins
 - ✓ Subqueries
 - ✓ Constraints
-

4.3 SQL Queries (SQL*Plus Compatible)

Query 1: Display all customers

(Simple SELECT)

```
SELECT * FROM customer;
```

Query 2: Display all available products

(WHERE condition)

```
SELECT product_name, price, stock_quantity  
FROM product  
WHERE availability_status = 'Available';
```

Query 3: Update stock quantity after an order

(DML – UPDATE)

```
UPDATE product  
SET stock_quantity = stock_quantity - 2  
WHERE product_id = 101;
```

Query 4: Delete a cancelled order

(DML – DELETE)

```
DELETE FROM orders  
WHERE order_status = 'Cancelled';
```

Query 5: Find total number of customers

(Aggregate Function)

```
SELECT COUNT(*) AS total_customers  
FROM customer;
```

Query 6: Find average price of products

(Aggregate Function)

```
SELECT AVG(price) AS average_price  
FROM product;
```

Query 7: Display number of orders placed by each customer

(GROUP BY)

```
SELECT customer_id, COUNT(order_id) AS total_orders
FROM orders
GROUP BY customer_id;
```

Query 8: Display customers who placed more than one order

(GROUP BY + HAVING)

```
SELECT customer_id, COUNT(order_id) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(order_id) > 1;
```

Query 9: Display customer names along with their order details

(JOIN)

```
SELECT c.customer_name, o.order_id, o.order_status, o.total_amount
FROM customer c
JOIN orders o
ON c.customer_id = o.customer_id;
```

Query 10: Display order details with product names

(Multiple table JOIN)

```
SELECT o.order_id, p.product_name, oi.quantity, oi.price_at_order
FROM orders o
JOIN order_item oi ON o.order_id = oi.order_id
JOIN product p ON oi.product_id = p.product_id;
```

Query 11: Display products priced higher than average price

(Subquery)

```
SELECT product_name, price
FROM product
WHERE price > (SELECT AVG(price) FROM product);
```

Query 12: Add a constraint to ensure valid order status

(DDL – ALTER with CONSTRAINT)

```
ALTER TABLE orders
ADD CONSTRAINT chk_order_status
CHECK (order_status IN ('Pending', 'Delivered', 'Cancelled'));
```

4.4 Output Verification

- All queries execute successfully in **SQL*Plus**
 - Correct and meaningful outputs are generated
 - Queries reflect real-world quick commerce operations
 - Functionalities are clearly demonstrated
-

OUTPUTS:-

QUERY 1

```
SQL> select * from customer;
```

CUSTOMER_ID	CUSTOMER_NAME	PHONE_NUMBER	EMAIL	ADDRESS	REGISTRATION_DATE
1	Amit Sharma	9876543210	amit@gmail.com	Mumbai	25-JAN-26
2	Neha Patil	9123456780	neha@gmail.com	Pune	25-JAN-26
3	Rahul Verma	9988776655	rahul@gmail.com	Delhi	25-JAN-26
4	Sneha Joshi	9012345678	sneha@gmail.com	Nagpur	25-JAN-26
5	Karan Mehta	9090909090	karan@gmail.com	Ahmedabad	25-JAN-26

QUERY 2

```
SQL> SELECT product_name, price, stock_quantity FROM product WHERE availability_status = 'Available';
```

PRODUCT_NAME	PRICE	STOCK_QUANTITY
Milk	50	100
Bread	40	60
Rice	70	200
Eggs	6	300
Soap	30	150

QUERY 3

```
SQL> UPDATE product SET stock_quantity = stock_quantity - 2 WHERE product_id = 101;
```

```
1 row updated.
```

```
SQL> select * from product;
```

PRODUCT_ID	PRODUCT_NAME	CATEGORY	PRICE	STOCK_QUANTITY	AVAILABILITY
101	Milk	Dairy	50	98	Available
102	Bread	Bakery	40	60	Available
103	Rice	Grocery	70	200	Available
104	Eggs	Poultry	6	300	Available
105	Soap	Personal Care	30	150	Available

QUERY 4

```
SQL> SELECT order_id FROM orders WHERE order_status = 'Cancelled';
```

ORDER_ID
1004

```
SQL> DELETE FROM order_item WHERE order_id = 1004;
```

```
1 row deleted.
```

```
SQL> DELETE FROM orders WHERE order_id = 1004;
```

```
1 row deleted.
```

QUERY 5

```
SQL> SELECT COUNT(*) AS total_customers FROM customer;
```

```
TOTAL_CUSTOMERS
-----
                5
```

QUERY 6

```
SQL> SELECT AVG(price) AS average_price FROM product;
```

```
AVERAGE_PRICE
-----
            39.2
```

QUERY 7

```
SQL> SELECT customer_id, COUNT(order_id) AS total_orders FROM orders GROUP BY customer_id;
```

```
CUSTOMER_ID TOTAL_ORDERS
-----
1             1
2             1
3             1
5             1
```

QUERY 8

```
SQL> SELECT customer_id, COUNT(order_id) AS order_count FROM orders GROUP BY customer_id HAVING COUNT(order_id) > 1;
```

```
no rows selected
```

QUERY 9

```
SQL> SELECT c.customer_name, o.order_id, o.order_status, o.total_amount FROM customer c JOIN orders o ON c.customer_id = o.customer_id;
```

CUSTOMER_NAME	ORDER_ID	ORDER_STATUS	TOTAL_AMOUNT
Amit Sharma	1001	Delivered	250
Neha Patil	1002	Pending	180
Rahul Verma	1003	Delivered	320
Karan Mehta	1005	Delivered	150

QUERY 10

```
SQL> SELECT o.order_id, p.product_name, oi.quantity, oi.price_at_order FROM orders o JOIN order_item oi ON o.order_id = oi.order_id JOIN product p ON oi.product_id = p.product_id;
```

ORDER_ID	PRODUCT_NAME	QUANTITY	PRICE_AT_ORDER
1001	Milk	2	50
1002	Bread	1	40
1003	Rice	3	70
1005	Soap	2	30

QUERY 11

```
SQL> SELECT product_name, price FROM product WHERE price > (SELECT AVG(price) FROM product);
```

PRODUCT_NAME	PRICE
Milk	50
Bread	40
Rice	70

QUERY 12

```
SQL> ALTER TABLE orders ADD CONSTRAINT chk_order_status CHECK (order_status IN ('Pending', 'Delivered', 'Cancelled'));
```

```
Table altered.
```

